Московский государственный университет имени М.В. Ломоносова Факультет вычислительной математики и кибернетики Кафедра автоматизации систем вычислительных комплексов

Ефанов Михаил Михайлович

Алгоритм управления перегрузкой на основе пропускной способности и задержки в условиях применения пакетной балансировки

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

Научный руководитель:
д.ф.-м.н., профессор, чл.-корр. РАН Смелянский Руслан Леонидович

Научный консультант:
к.ф.-м.н.,ассистент Степанов Евгений Павлович

Москва, 2026

Аннотация

Работа посвящена разработке модификации алгоритма управления перегрузкой BBR для эффективной работы в условиях пакетной балансировки. Пакетная балансировка повышает производительность сетевых сервисов за счёт горизонтального масштабирования, однако классические алгоритмы управления перегрузкой не адаптированы для совместной работы с ней. Основные проблемы включают переупорядочивание пакетов и деградацию отдельных каналов, что приводит к ложным сигналам перегрузки и снижению скорости передачи данных. В работе проведён обзор существующих решений по повышению устойчивости алгоритмов управления перегрузкой к переупорядочиванию пакетов и деградации каналов: TCP-NCR, Lazy TCP и MSwift. На основе сравнительного анализа сделан вывод, что ни один из рассмотренных подходов не может быть непосредственно применён для модификации BBR. В результате предложено и экспериментально обосновано решение, заключающееся в замене фиксированного порога быстрой ретрансляции динамическим значением, вычисляемым на основе оценки пропускной способности и задержки соединения.

Оглавление

Введение . . . 3
1. Постановка задачи . . . 4
2. Обзор существующих решений . . . 5
   2.1. TCP-NCR . . . 6
   2.2. Lazy TCP (LTCP) . . . 7
   2.3. MSwift . . . 7
   2.4. Вывод . . . 8
3. Алгоритм работы BBR . . . 9
4. Исследование работы BBR в условиях пакетной балансировки . . . 12
   4.1. Топология сети . . . 12
   4.2. Исследование скорости передачи данных в условиях симметричных маршрутов . . . 13
   4.3. Исследование скорости передачи данных в условиях ассимметричных маршрутов . . . 15
   4.4. Вывод . . . 17
5. Адаптация BBR к условиям пакетной балансировки . . . 17
Литература . . . 19

Введение

Современные компьютерные сети характеризуются высокой динамикой нагрузки, неоднородностью каналов и всё более широким применением многопутевой передачи. На практике трафик одного транспортного соединения может проходить по нескольким параллельным маршрутам — как из-за особенностей маршрутизации и инженерии трафика в сети, так и из-за применения механизмов балансировки нагрузки. При этом балансировка может выполняться не только по потокам, но и на уровне отдельных пакетов (per-packet load balancing, packet spraying), когда последовательные пакеты одного и того же соединения целенаправленно распределяются по разным путям. Такая стратегия потенциально повышает использование доступной пропускной способности совокупности путей и улучшает устойчивость передачи к локальным перегрузкам и отказам, но одновременно меняет свойства наблюдаемых транспортом сигналов состояния сети [1].

Классические алгоритмы управления перегрузкой TCP исторически опираются на интерпретацию потери пакета (loss-based) как основного признака перегрузки. Эта эвристика была разумной в условиях ограниченных буферов и относительно простых путей, однако в современных сетях приводит к систематическим проблемам. При больших буферах на узких местах loss-based управление перегрузкой склонно формировать устойчивые очереди и увеличивать задержки, а при потерях, не связанных с перегрузкой, — неоправданно снижать скорость, недоиспользуя доступную ёмкость канала. На этом фоне получили развитие подходы, основанные не на потерях, а на моделировании физических ограничений пути: пропускной способности узкого места (BtlBw) и задержки распространения (RTprop). Алгоритм BBR является представителем этого направления: он оценивает BtlBw и RTprop, стремится поддерживать объём данных «в полёте» (отправленные пакеты на, которые еще не получен ACK) около BDP = BtlBw × RTprop, а темп передачи задаёт через задержку между отправкой следующего пакета (pacing). Такой подход позволяет одновременно добиваться высокой полезной пропускной способности и удерживать задержки на низком уровне, что подтверждается опытом крупномасштабного применения в реальных сервисах и на разнообразных типах сетей, включая мобильные [2].

Однако применение многопутевой передачи и особенно пакетной балансировки усложняет задачу управления перегрузкой. Транспортный протокол перестаёт наблюдать «один путь» с единственными характеристиками задержки и пропускной способности; вместо этого он получает поток обратной связи, являющийся смесью нескольких путей, которые могут различаться по задержкам, текущим очередям и доступной скорости. Даже если изначально пути близки по параметрам, в реальной сети неизбежны временные асимметрии: локальные перегрузки, изменения маршрута, сбои и деградации отдельных каналов, приводящие к увеличению задержки или потере части пакетов на одном из путей. В таких условиях пакеты могут приходить не по порядку, что порождает дубликатные ACK и другие аномальные наблюдения, которые традиционный TCP склонен трактовать как перегрузку. Это приводит к неоправданному снижению скорости передачи и деградации производительности — иногда до состояния, которое можно описать как коллапс пропускной способности при наличии даже небольшой доли

перегруженных путей из общего числа путей, между которыми реализована балансировка пакетов [3].

Проблема устойчивости TCP к переупорядочению пакетов рассмотрена в [4]: в этой работе введено понятие «устойчивости к неперегрузочным событиям» (Non-Congestion Robustness) и предложен подход, в котором порог срабатывания быстрой ретрансляции по duplicate ACK не фиксирован, а динамически увеличивается в зависимости от состояния соединения. Это позволяет лучше отличать истинную потерю от переупорядочения и уменьшает число избыточных повторных передач, хотя вводит компромисс в виде замедления реакции на реальную перегрузку. В работе [5] предложены более специализированные механизмы: Lazy TCP откладывает реакцию на dupACK («lazy trigger») и по-разному реагирует на единичные и последовательные потери, стремясь не снижать скорость из-за событий, которые с высокой вероятностью не являются перегрузкой.

Однако устойчивости только к переупорядочиванию пакетов часто недостаточно. Современные алгоритмы управления перегрузкой используют разные сигналы, например задержку (delay-based). В условиях пакетной балансировки эти сигналы начинают поступать как смесь измерений по разным путям и могут содержать выбросы из-за перегруженного или более длинного пути. Показано, что для широкого класса алгоритмов, использующих AIMD-подобные принципы, при наличии перегруженного пути с вероятностью попадания на него q пропускная способность может ухудшаться по закону порядка $\Theta(1/\sqrt{q})$; причём это проявляется для различных CCA и сохраняется даже у вариантов, устойчивых к переупорядочению [3]. В качестве более общего принципа предложено применять устойчивую к подобным выбросам агрегацию многопутевых сигналов, например использовать медиану по окну недавних измерений вместо реакции на «последнее» наблюдение [3].

Таким образом, актуальность темы данной дипломной работы определяется следующими обстоятельствами:

- распространённостью и практической значимостью алгоритмов управления перегрузкой, ориентированных на пропускную способность и задержку, таких как BBR, позволяющих приблизиться к высокой утилизации канала при малых очередях [2];
- растущей ролью пакетной балансировки и других форм многопутевой доставки, которые могут повышать использование доступной суммарной ёмкости, но провоцируют переупорядочение и делают транспортные измерения смешанными по нескольким путям [1];
- наличием фундаментальных эффектов деградации производительности CCA в многопутевом per-packet сценарии и необходимостью более робастной обработки сигналов перегрузки, чем простая защита от dupACK [3];
- существованием частичных решений (динамический порог dupACK, «ленивые» реакции), которые полезны, но не закрывают полностью задачу устойчивого управления при многопутевой смешанной обратной связи [3–5]

1. Постановка задачи

Цель.

Повышение скорости транспортного потока за счет алгоритма управления перегрузкой BBR, адаптированного к применению в условиях пакетной балансировки.

Дано.

1. Конечное, упорядоченное множество пакетов, отправленных до настоящего момента $P = \{p_1, p_2, ..., p_n\}$
2. Размер сегмента MSS.
2. Соответствующее ему упорядоченное множество $\mathrm{RTT} = \{\mathrm{RTT}_1, \mathrm{RTT}_2, ..., \mathrm{RTT}_k\}$, где $k \leq n$, так как не на все пакеты пришло подтверждение и измерен RTT.
3. Число последовательно полученных дублирующих ACK для одного и того же неподтверждённого пакета d.
4. Оценка пропускной способности узкого места в настоящий момент BtlBw.
5. Оценка минимальной задержки распространения в настоящий момент RTprop.

Требуется.

Построить отображение:

$$F : (\mathrm{RTT}, d, \mathrm{BtlBw(t)}, \mathrm{RTprop(t)}) \to \{\mathrm{LOSS}, \mathrm{REORDERING}\}$$

которое по наблюдаемым величинам, определяет тип события:

LOSS - потеря пакета, вызванная перегрузкой сети или ошибкой на одном из соединений.

$\mathrm{REORDERING}$ - переупорядочивание пакетов.

Задачи:

1. Провести обзор существующих алгоритмов управления перегрузок, адаптированных к применению в условиях пакетной балансировки.
3. Разработать решение - модификацию BBR, направленную на повышение производительности в условиях пакетной балансировки.
4. Провести исследование скорости транспортного потока, управляемого разработанным алгоритмом, в условиях пакетной балансировки

2. Обзор существующих решений

Цель данного обзора состоит в том, чтобы определить, какие из существующих подходов к обеспечению устойчивости TCP к пакетной балансировке могут быть использованы как основа для модификации BBR. Для этого рассматриваются не только сами механизмы защиты от переупорядочивания и деградации отдельных путей, но и степень их совместимости с моделью управления перегрузкой, лежащей в основе BBR.

Сравнение решений далее проводится по следующим критериям.

- **Устойчивость к переупорядочиванию пакетов.** Этот критерий показывает, способен ли алгоритм не интерпретировать обычное для пакетной балансировки нарушение порядка доставки как сигнал потери. Он важен, поскольку ложные срабатывания fast retransmit приводят к избыточным повторным передачам и снижают полезную пропускную способность.

- **Устойчивость к деградации отдельных путей.** Здесь оценивается, насколько хорошо подход справляется с ситуацией, когда лишь часть маршрутов становится хуже по задержке, потерям или пропускной способности на короткий период. Этот критерий важен для нашей работы, потому что в реальных условиях пакетной балансировки пути редко остаются полностью постоянными, и алгоритм должен быть устойчив именно к смешанной обратной.
- **Независимось от стратегии балансировки и колличества маршрутов.** В задаче нет ограничений на стратегию балансировки в сети. Алгоритм не должен полагаться ни на какие предположения о том, как происходит балансировка пакетов между маршрутами, сколько существует маршрутов для балансировки, и просиходит ли балансировка вообще.
- **Совместимость с моделью BBR.** Под этим понимается, можно ли встроить рассматриваемый механизм в BBR без отказа от его базовых принципов — оценок `BtlBw` и `RTprop`, управления через `pacing_rate` и model-based логики в целом. Этот критерий является ключевым, поскольку цель работы состоит не в замене BBR другим алгоритмом, а в разработке его модификации для условий пакетной балансировки.

2.1. TCP-NCR

TCP-NCR (Non-Congestion Robustness) был предложен как модификация классического TCP, в котором запуск fast retransmit и последующее уменьшение окна перегрузки тесно связаны с получением серии duplicate ACK. Основная идея метода состоит в том, чтобы сделать порог срабатывания не фиксированным (`3 dupACK`), а зависящим от текущего состояния соединения: чем больше данных находится “в полёте”, тем больший уровень переупорядочивания допускается без немедленного объявления потери [4].

Порог задаётся выражением

$$\mathrm{DupThresh} = \max\left(\mathrm{LT_F} \cdot \frac{\mathrm{InFlightSize}}{\mathrm{MSS}}, 3\right)$$

где `InFlightSize` — текущий объём неподтверждённых данных, `MSS` — размер сегмента, а `LT_F` — параметр настройки алгоритма. Дополнительно TCP-NCR использует алгоритм Extended Limited Transmit: при поступлении duplicate ACK отправитель продолжает осторожно вводить в сеть новые сегменты, чтобы сохранить самотактирование ACK-потоком и не остановить передачу до накопления достаточного числа подтверждений.

Такой подход действительно полезен для loss-based TCP, поскольку в нём duplicate ACK являются одним из центральных сигналов, определяющих и факт потери, и момент снижения `cwnd`. Однако для BBR этот механизм концептуально плохо подходит. Как показано в разделе Section 3, BBR не использует правило вида «получили `k` duplicate ACK — уменьшаем окно и скорость». Его основное управление строится вокруг модели пути, включающей оценки `BtlBw` и `RTprop`, а темп передачи задаётся через `pacing_rate`, а не через непосредственную реакцию на число duplicate ACK. Следовательно, динамическая настройка `DupThresh` затрагивает лишь периферийный механизм обнаружения потерь, но не влияет на главный источник ошибок BBR при пакетной балансировке — искажение измерений скорости доставки и RTT из-за смешения наблюдений от нескольких путей. Поэтому TCP-NCR можно рассматривать лишь как локальную защиту от ложного fast

retransmit, но не как подходящую основу для модификации BBR в условиях пакетной балансировки.

2.2. Lazy TCP (LTCP)

Lazy TCP также разрабатывался для сценария packet spraying, но, как и TCP-NCR, остаётся привязанным к классической TCP-логике управления через потери и окно перегрузки. В работе [5] предложены два основных механизма.

Первый механизм — **lazy trigger**: отправитель не реагирует немедленно на серию duplicate ACK, а откладывает запуск восстановления, чтобы дать переупорядоченным пакетам возможность прийти без ложного уменьшения скорости. Второй механизм — **stubborn window**: алгоритм различает **дискретные** и **последовательные** потери. Если по информации `SACK` видно, что потери единичны и не образуют длинную непрерывную последовательность, LTCP трактует их как следствие деградации отдельного пути и не уменьшает `cwnd`; если же потери выглядят как серия подряд идущих сегментов, они интерпретируются как признак перегрузки, и окно уменьшается. Для различения типов потерь используется `SACK` (Selective ACK) — это опция TCP, в которой получатель сообщает, какие блоки данных он уже получил; по ней отправитель может понять, какие сегменты отсутствуют. Критерий «последовательности» задаётся порогом `k`: если потеряно больше `k` пакетов подряд, потери считаются связанными с перегрузкой (successive loss), иначе — дискретными. `k` трактуется как ожидаемое количество соседних пакетов, которые могли попасть на один перегруженный канал при распылении.

Для классического TCP такая логика естественна, потому что именно `cwnd` является главным регулятором интенсивности передачи. Но в BBR окно перегрузки играет вспомогательную роль: оно ограничивает верхнюю границу `inflight`, тогда как фактическая скорость передачи определяется `pacing_rate`, вычисляемым из оценки `BtlBw`. Поэтому перенос идеи **stubborn window** в BBR не даёт ожидаемого эффекта. Даже если не уменьшать `cwnd` при «дискретных» потерях, BBR всё равно будет принимать решения на основе наблюдаемой скорости доставки и RTT. Кроме того, классификация потерь на дискретные и последовательные в LTCP опирается на предположение о количестве подряд идущих пакетов, попавших в один канал при распылении; в реальных сетях эту величину трудно оценить заранее и она может меняться во времени. По этим причинам LTCP также не подходит как направление модификации BBR.

2.3. MSwift

В работе [3] рассматривается не только проблема duplicate ACK, но и общая задача обработки сигналов перегрузки при packet spraying. В MSwift наряду с механизмами LTCP используется медианная агрегация задержки: вместо реакции на отдельное измерение RTT алгоритм вычисляет медиану по окну последних наблюдений,

$$N = \max(\min(\alpha \cdot \mathrm{cwnd}, K), 3)$$

, и затем принимает решение об уменьшении окна, если медианная задержка превышает заданный порог.

Однако и этот подход не переносится на BBR напрямую. Причина состоит в том, что MSwift остаётся delay-based вариантом семейства Swift/AIMD, где управляющее

воздействие формулируется как реакция на агрегированную задержку: если задержка слишком велика, `cwnd` уменьшается, обычно мультипликативно. BBR устроен принципиально иначе. Во-первых, он не использует пороговую схему вида «RTT выше порога — уменьшить окно вдвое»; во-вторых, его оценка `RTprop` по смыслу является не типичным, а минимальным RTT на длинном окне, то есть попыткой выделить задержку распространения без очереди. Замена этой оценки на медиану разрушила бы сам физический смысл параметра `RTprop`: медиана отражает типичное состояние смеси путей и очередей, а BBR требуется нижняя опорная оценка, близкая к базовой задержке.

Аналогичная проблема возникает и для оценки пропускной способности. В BBR параметр `BtlBw` строится как максимум наблюдаемой `delivery_rate` на скользящем окне, поскольку алгоритм стремится не усреднить скорость, а выделить устойчивую верхнюю оценку доступной пропускной способности узкого места. Робастная агрегация по медиане, полезная для подавления выбросов в delay-based алгоритме, здесь меняет саму цель оценивания: вместо оценки потенциально достижимой скорости получится «типичная» скорость смеси путей, что приведёт к систематическому занижению модели и недоиспользованию канала. Следовательно, MSwift важен как указание на сам факт необходимости робастной обработки многопутевых измерений, но его конкретный механизм — медиана RTT с последующим уменьшением `cwnd` — несовместим с базовыми принципами работы BBR.

2.4. Вывод

	TCP-NCR	LTCP	MSwift
Отсутствует снижение скорости при переупорядочивании пакетов	Да	Да	Да
Отсутствует снижение скорости при ассиметрии каналов	Нет	Да	Да
Независимость от стратегии балансировки	Да	Нет	Нет, но можно настроить оптимально для большинства случаев
Применимость к BBR	Нет	Нет	Нет

Таблица 1. Сравнение подходов TCP-NCR, LTCP и MSwift

Рассмотренные подходы решают разные частные проблемы packet spraying, однако ни один из них не может быть непосредственно использован как модификация BBR. TCP-NCR и LTCP ориентированы на классический TCP, в котором ключевыми управляющими событиями являются duplicate ACK, fast retransmit и изменение `cwnd` после потерь. Для BBR такие механизмы вторичны: его поведение определяется моделью пути, оценками `BtlBw` и `RTprop`, а также управлением через pacing. Поэтому улучшение обработки duplicate ACK само по себе не устраняет главную причину деградации BBR при пакетной балансировке — искажение измерений, из которых строится модель.

MSwift поднимается на более общий уровень и показывает, что при многопутевой доставке требуется робастная обработка сигналов. Тем не менее его конкретная реализация также не подходит для BBR, поскольку основана на медианной оценке RTT и пороговом уменьшении `cwnd`, тогда как BBR нуждается в оценках другого типа: минимальной задержки распространения и верхней оценки пропускной способности. Прямая замена этих оценок медианой или перенос AIMD-реакции на задержку привели бы не к адаптации BBR, а к фактической подмене его модели другим алгоритмом.

Следовательно, направление модификации BBR должно опираться не на прямое заимствование TCP-NCR, LTCP или MSwift, а на сохранение базовой модели BBR при разработке специальных, совместимых с ней методов интерпретации многопутевых измерений. Иначе говоря, задача состоит в том, чтобы сделать оценки `RTprop`, `BtlBw` и связанных с ними величин устойчивыми к переупорядочиванию и смешению сигналов от нескольких путей [2–5].

3. Алгоритм работы BBR

Алгоритм BBR (Bottleneck Bandwidth and Round-trip propagation time) относится к классу model-based алгоритмов управления перегрузкой: его работа основана не на прямой реакции на потери пакетов, а на построении явной модели пути передачи. В основе BBR лежит идея, восходящая к результатам Леонарда Клейнрока [6]: скорость отправки пакетов в сеть должна обеспечивать максимально возможную пропускную способность при минимально необходимом уровне очередей в сети.

В рамках этой модели BBR непрерывно оценивает два ключевых параметра пути:

- минимальную задержку распространения `RTprop`, приближаемую наблюдаемым минимумом времени обхода пакета туда и обратно;
- пропускную способность узкого места `BtlBw`, оцениваемую по скорости доставки данных получателю.

Идеальный объём данных в сети при этом определяется произведением пропускной способности на задержку распространения, то есть bandwidth-delay product:

$$\mathrm{BDP} = \mathrm{BtlBw} \cdot \mathrm{RTprop}$$

Величина `BDP` задаёт естественную целевую оценку числа данных «в полёте» (`inflight` - отправленные пакеты, на которые еще не был получен ACK), необходимого для насыщения канала без формирования избыточной очереди. Соответственно, BBR управляет соединением с помощью двух взаимосвязанных механизмов:

- регулирования темпа отправки пакетов (`pacing`);
- ограничения окна перегрузки `cwnd`, которое ограничивает максимально допустимый объём неподтверждённых данных.

Темп передачи выбирается пропорционально текущей оценке пропускной способности узкого места:

$$\mathrm{pacing_rate} = \mathrm{pacing_gain} \cdot \mathrm{BtlBw}$$

Здесь `pacing_gain` — коэффициент, зависящий от текущего состояния автомата BBR и определяющий, должен ли отправитель ускорять передачу, поддерживать стационарный режим или временно снижать интенсивность для дренирования очереди.

Оценка `BtlBw` формируется по наблюдаемой скорости доставки данных. В базовой версии BBR она вычисляется как максимум скорости передачи (`delivery rate`) на скользящем интервале длиной порядка десяти раундов передачи:

$$\mathrm{BtlBw} = \max_{t \in W} \mathrm{delivery_rate}(t), \quad |W| \approx 10 \cdot \mathrm{RTprop}$$

Тем самым BBR стремится выделить устойчивую верхнюю оценку доступной пропускной способности, подавляя кратковременные просадки. Оценка `RTprop`, напротив, выбирается как минимум RTT на достаточно длинном временном окне, поскольку именно минимальное наблюдаемое значение наилучшим образом соответствует задержке распространения без влияния очередей:

$$\mathrm{RTprop} = \min_{t \in W_r} \mathrm{RTT}(t)$$

Таким образом, базовая логика BBR может быть интерпретирована как поддержание условия

$$\mathrm{inflight} \approx \mathrm{BDP}$$

то есть такого режима, при котором соединение заполняет канал, но по возможности не создаёт устойчивой очереди в узком месте.

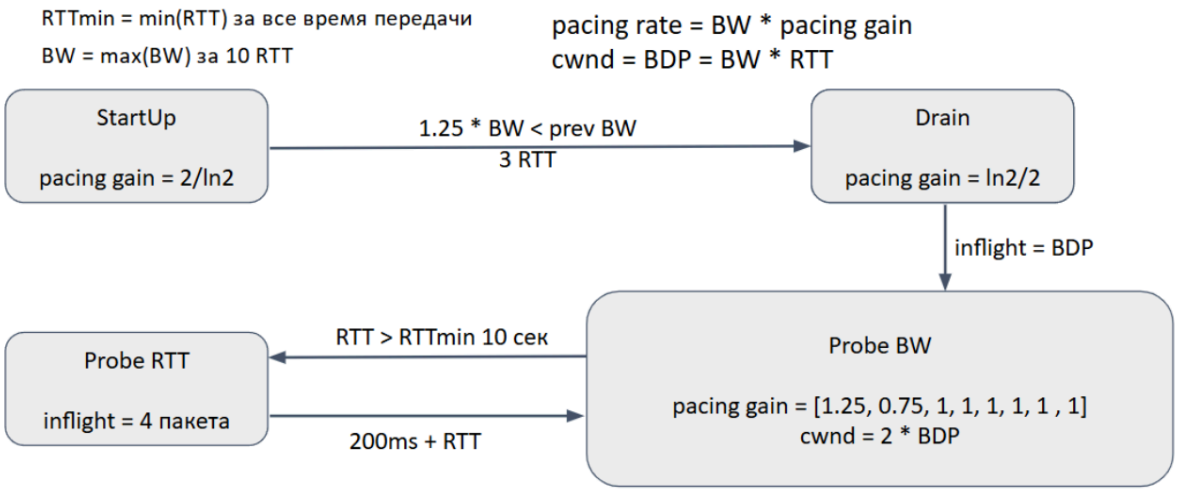

Рисунок 1. Схема автомата BBR.

Функционирование BBR удобно описывать как работу конечного автомата из четырёх основных состояний [2].

Состояние StartUp. На начальном этапе отправитель стремится быстро приблизиться к доступной пропускной способности пути. Для этого используется повышающий коэффициент темпа передачи

$$\mathrm{pacing_gain} = \frac{2}{\ln 2}$$

что соответствует экспоненциальному наращиванию интенсивности отправки до тех пор, пока оценки скорости доставки продолжают заметно расти. Переход из состояния

StartUp выполняется тогда, когда прирост оценки `BtlBw` перестаёт быть существенным в течение нескольких последовательных раундов передачи; в реализации BBR это интерпретируется как достижение насыщения узкого места.

Состояние Drain. После агрессивного заполнения канала, характерного для StartUp, в сети обычно формируется избыточная очередь. Поэтому на этапе Drain отправитель временно снижает темп передачи, чтобы удалить накопившиеся данные из буфера и вернуть соединение к режиму, близкому к целевому `BDP`. Для этого используется коэффициент

$$\text{pacing_gain} = \frac{\ln 2}{2}$$

Переход к следующему состоянию происходит, когда объём данных в сети становится не больше целевого значения, то есть выполняется условие

$$\text{inflight} \leq \text{BDP}$$

Состояние ProbeBW. Это основное рабочее состояние алгоритма, в котором соединение проводит большую часть времени. Его задача состоит в периодическом зондировании доступной пропускной способности: отправитель чередует фазы умеренного ускорения, умеренного замедления и нейтральной передачи. В классическом варианте BBR используется циклическая последовательность коэффициентов

$$(1.25, 0.75, 1, 1, 1, 1, 1, 1)$$

с переключением между фазами примерно один раз за RTT. Фаза с коэффициентом больше единицы позволяет проверить, не увеличилась ли доступная пропускная способность, а фаза с коэффициентом меньше единицы способствует уменьшению возможной очереди. В этом состоянии окно перегрузки обычно выбирается с запасом относительно `BDP` (`cwnd = 2 dot BDP`), чтобы pacing оставался основным механизмом управления скоростью, а `cwnd` не ограничивал передачу преждевременно.

Состояние ProbeRTT. Поскольку минимальная задержка распространения может со временем устаревать из-за постоянного присутствия очереди, BBR периодически переходит в режим переоценки `RTprop`. Для этого объём данных в сети искусственно уменьшается, а окно перегрузки сокращается до малого значения (в классической реализации — до четырёх сегментов), чтобы дать очередям возможность полностью опустеть. После короткого интервала наблюдения, достаточного для измерения нового минимального RTT, автомат возвращается в состояние ProbeBW [2].

Реакция на `3 DupAck`. BBR не изменяет свое поведение при получении `DupAck` пакетов. Однако, когда отправитель получает `3 DupAck` на один пакет, то пакет считается потерянным и повторно отправляется в сеть. Этот механизм называется `Fast Recovery` и не зависит от алгоритма управления перегрузкой.

Реакция на кратковременные деградации каналов. Из рассмотренного выше алгоритма становится ясно, что BBR не реагирует на измерения, несвойственные для канала, такие как повышение RTT в для отдельных сегментов.

4. Исследование работы BBR в условиях пакетной балансировки

Для исследования поведения алгоритма управления перегрузкой в условиях применения пакетной балансировки требуется экспериментальное исследование в среде, позволяющей изолированно изучать указанные в выше эффекты. В качестве основного инструмента выбран симулятор сетей NS-3 версии 3.45 (Network Simulator 3)[7]. Этот выбор обусловлен необходимостью детерминированного воспроизведения сценариев с контролируемыми параметрами задержки и пропускной способности на каждом из балансируемых путей, а также наличием корректной реализации BBR [8,9].

4.1. Топология сети

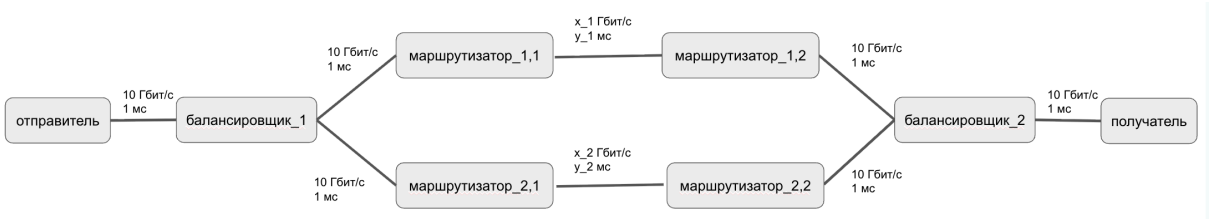

Рисунок 2. Топология сети в эксперименте.

Для экспериментального исследования был выбран сценарий, моделирующий передачу данных между одним отправителем и одним получателем через два последовательных уровня пакетной балансировки. Топология сети, изображённая на рисунке, включает в себя следующие узлы: источник трафика (отправитель), два балансировщика нагрузки (балансировщик_1 и балансировщик_2), два параллельных маршрута, а также конечный узел-получатель (получатель). Отправитель соединён с первым балансировщиком, который распределяет поступающие пакеты по N параллельным маршрутам. Каждый из этих маршрутов содержит один роутер, после чего все потоки повторно агрегируются вторым балансировщиком и направляются получателю. Колличество парраллельных маршрутов = 2. Ограничение обусловлено тем, что в большинстве топологий, использующих балансировку (по потокам или по пакетам), колличество парраллельных маршрутов не превосходит 3 [10]. Различные настройки параметров соединений между маршрутизаторами позволяют моделировать симметричные и ассиметричные парраллельные маршруты.

Участок сети	Пропускная способность	Задержка
отправитель - балансировщик$_1$	10 Gbps	1 ms
балансировщик$_1$ - маршрутизатор	10 Gbps	1 ms
маршрутизатор$_{1,1}$ - маршрутизатор$_{1,2}$	x_1 Mbps	y_1 ms
маршрутизатор$_{2,1}$ - маршрутизатор$_{2,2}$	x_2 Mbps	y_2 ms
маршрутизатор - балансировщик$_2$	10 Gbps	1 ms
балансировщик$_2$ - получатель	10 Gbps	1 ms

Таблица 2. Параметры соединений в эксперменте

Параметры всех сетевых соединений приведены в таблице 2. Участок отправитель-балансировщик$_1$ и все последующие участки, характеризуются высокой пропускной способностью (10 Гбит/с) и фиксированной задержкой в 1 мс, что исключает ограничение скорости передачи данных в этих соединениях. Параметры множества параллельных каналов маршрутизатор$_{i,1}$ - маршрутизатор$_{i,2}$, $i \in [1, 2]$ задают условия эксперимента для каждого маршрута. Для каждого такого канала индивидуально определяются два ключевых показателя: пропускная способность (data_rate) и задержка (delay). Размер очереди для всех каналов установлен удвоенным произведением пропускной способности на задержку ($2 \cdot \mathrm{data_rate} \cdot \mathrm{delay} = 2 \cdot \mathrm{BDP}$), что соответствует оптимальному значению для буферов при использовании алгоритма BBR в транспортном агенте [11].

Для моделирования балансировки, написан класс L3 балансировщик, использующий стратегию round robin при балансировке. При маршрутизации Ack пакетов, также происходит балансировка, причем Ack может пойти не по тому маршруту, по которому шел пакет.

4.2. Исследование скорости передачи данных в условиях симметричных маршрутов

Первая серия экспериментов проведена для оценки поведения алгоритма BBR в идеализированных условиях, когда все параллельные пути между балансировщиками обладают идентичными характеристиками. Симметричная конфигурация (при $N = 2$) позволяет установить базовую линию производительности BBR при пакетной балансировке, исключив влияние асимметрии задержек или пропускной способности.

В данной серии экспериментов параметры каждого из параллельных каналов `балансировщик_1 – роутер_i` заданы следующим образом:

- пропускная способность (`data_rate`) — от 50 `Мбит/с` до 1 `Гбит/с` ;
- задержка распространения (`delay`) — 10 `мс`;

На рисунке представлен график изменения среднего значения полезной пропускной способности (goodput) во времени для базового алгоритма BBR при работе через два симметричных параллельных пути.

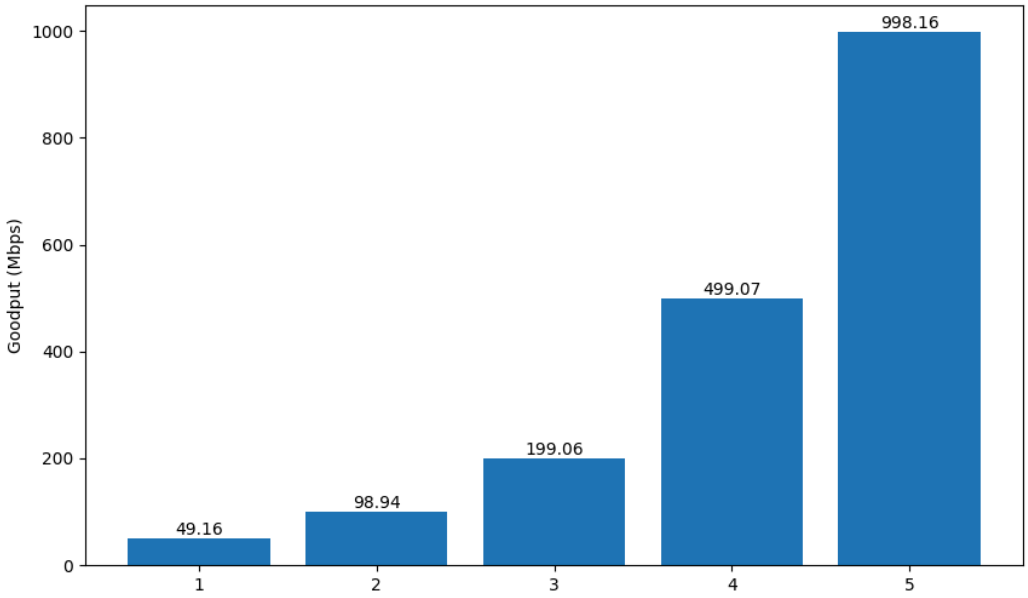

Рисунок 3. Изменение общей полезной пропускной способности BBR во времени при симметричных параллельных путях с скоростью передачи: 1 - 50 Мбит/с, 2 - 100 Мбит/с, 3 - 200 Мбит/с, 4 - 500 Мбит/с, 5 - 1 Гбит/с.

Ключевое наблюдение: несмотря на наличие двух параллельных путей в 1 эксперименте с суммарной ёмкостью 100 Мбит/с, измеренный общий goodput на протяжении всего эксперимента устойчиво держится на уровне 49-50 Мбит/с. Это значение лишь незначительно отличается от пропускной способности одного пути (50 Мбит/с) и составляет менее 50% от суммарной ёмкости двух каналов, такое же наблюдение и в остальных экспериментах.

Интерпретация полученного результата. Наблюдаемое поведение свидетельствует о том, что BBR в условиях пакетной балансировки не способен эффективно использовать совокупную пропускную способность параллельных путей даже при их полной симметрии. Алгоритм ведёт себя так, как будто в сети есть только один путь с пропускной способностью около 50 Мбит/с.

Причина этого эффекта кроется в механизме оценки пропускной способности узкого места (BtlBw), используемом BBR. Как было описано в разделе , оценка BtlBw формируется как максимум наблюдаемой скорости доставки (delivery_rate) на скользящем интервале. При пакетной балансировке пакеты распределяются между двумя путями, и скорость доставки, измеряемая по возвращаемым ACK, ограничивается пропускной способностью каждого отдельного пути, поскольку ACK возвращаются по тем же маршрутам, что и данные. BBR фиксирует максимальную наблюдаемую скорость, которая в симметричном случае равна 50 Мбит/с. Соответственно, pacing_rate устанавливается на уровне около 50 Мбит/с, и скорость передачи не превышает этого значения.

Стабильность режима. График демонстрирует практически идеальную горизонтальную линию без осцилляций и просадок. BBR быстро входит в состояние ProbeBW и остаётся в нём на протяжении всего эксперимента, поддерживая постоянный темп передачи. Отсутствуют как периодические снижения скорости (свойственные состоянию ProbeRTT), так как на протяжении всего времени передачи сохраняется низкий rtt.

Вывод по симметричному сценарию. Эксперимент в симметричной конфигурации демонстрирует, что базовый BBR не способен утилизировать суммарную пропускную способность двух параллельных каналов при пакетной балансировке. Наблюдаемый в 1 эксперименте goodput (49 - 50 Мбит/с) практически соответствует пропускной способности одного пути и составляет лишь половину от доступной совокупной ёмкости сети (100 Мбит/с). Этот результат служит важной отправной точкой для последующего анализа: он показывает, что проблема деградации BBR при пакетной балансировке проявляется даже в отсутствие асимметрии путей и заключается в фундаментальной неспособности алгоритма агрегировать пропускную способность нескольких каналов. В следующих разделах будет исследовано, как эта проблема усугубляется при появлении асимметрии задержек и потерь на отдельных путях.

4.3. Исследование скорости передачи данных в условиях ассимметричных маршрутов

Во второй серии экспериментов исследовано поведение алгоритма BBR при наличии асимметрии между параллельными путями. В отличие от симметричной конфигурации, где все пути обладали идентичными характеристиками (пропускная способность 50 Мбит/с, задержка 10 мс), в асимметричных сценариях один из путей имеет худшие параметры, а другой сохраняет эталонные значения. Такая конфигурация моделирует реальную сетевую ситуацию, когда различий в физических каналах или особенностей маршрутизации отдельные пути деградируют.

Цель асимметричных экспериментов — проверить гипотезу о том, что отдельный замедленный путь (с большей задержкой) может исказить измерения BBR настолько, что пропускная способность всего соединения снизится до уровня этого плохого пути или даже ниже. В каждом эксперименте задействовано два параллельных пути: канал 1 — эталонные с пропускной способностью 50 Мбит/с и задержкой 10 мс; канал 2 — урезанный, у которого варьируется один из параметров. Для чистоты эксперимента асимметрия вводится только по одному измерению за раз.

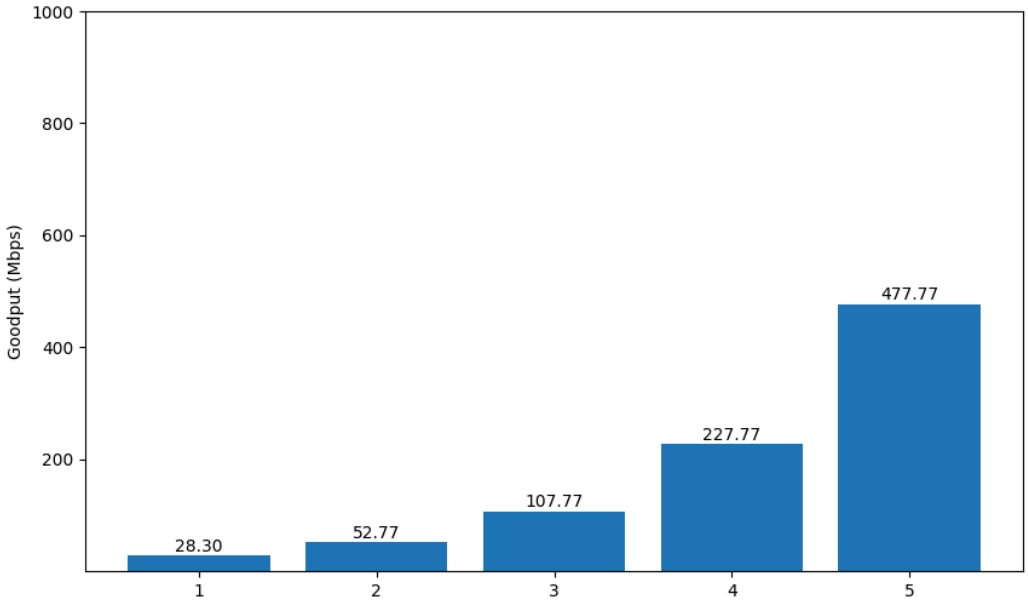

Рисунок 4. Изменение средней полезной пропускной способности BBR во времени при асимметричных параллельных путях с скоростью передачи: 1 - 50 Мбит/с, 2 - 100 Мбит/с, 3 - 200 Мбит/с, 4 - 500 Мбит/с, 5 - 1 Гбит/с. Задержка на 1 пути 10мс, на 2 - 20мс

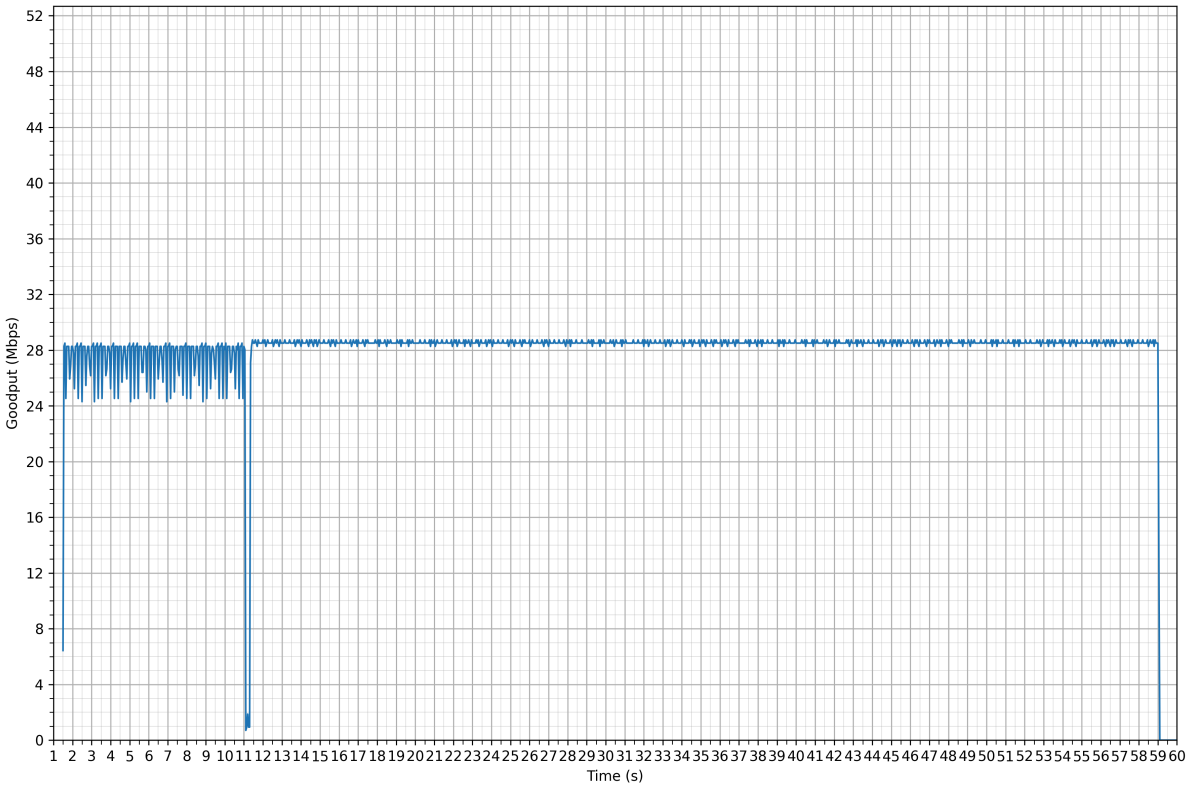

Рисунок 5. Изменение общей полезной пропускной способности BBR во времени при асимметричных параллельных путях с скоростью передачи 50 Мбит/с. Задержка на 1 пути 10 мс, на 2 пути 20 мс

Результаты эксперимента. В 1 эксперименте при задержке второго пути 20 мс установившаяся полезная пропускная способность (goodput) соединения составляет 28 Мбит/с. Во всех конфигурациях наблюдается характерный переходный процесс в начале передачи: в первые 10 секунд происходит кратковременное падение скорости, после чего следует фаза освобождения буфера, и затем соединение выходит на установившийся режим с указанными выше значениями goodput. Для остальных экспериментов наблюдается такое же поведение потока.

Вывод по сценарию с повышенной задержкой. Эксперименты демонстрируют, что BBR чувствителен к асимметрии задержек при пакетной балансировке. Увеличение задержки на одном из путей приводит к пропорциональному снижению установившейся полезной пропускной. Причина деградации заключается в том, что из-за сильного переупорядочивания пакетов отправитель посылает пакеты, попавшие на маршрут с большей задержкой, отправляются повторно до тех пор как не попадут на эталонный маршрут (это происходит из-за стандартного механизма Fast Retransmit, который повторно отправляет пакет при получении 3 DupAck), чтобы их ACK пришел в нужном порядке или пока не дойдет ACK первого из дублирующихся пакетов. Таким образом поток занимает 2 маршрута множеством дубликатов пакетов, полоса пропускания занимается ненужными данными и скорость полезной передачи падает.

4.4. Вывод

Ассиметрия маршрутов для балансировки не приводит к катастрофическим потерям, но постепенно увеличивает степень переупорядочивания пакетов, вследстие чего канал заполняется дубликатами пакетов. Однако даже при симметричных маршрутах, скорость транспортного потока не вырастает относительно скорости транспортного потока через 1 маршрут. Таким образом выявлены 2 проблемы использования BBR в сети с пакетной балансировкой:

1. Скорость передачи равна максимальной скорости передачи среди параллельных маршрутов, что меньше чем сумма по всем параллельным маршрутам.
2. Разлизие задержки на параллельных маршрутах понижает скорость передачи, даже при наличие небольшой разницы (в 2 раза).

В данной работе поставлена задача достижения устойчивости к переупорядочиванию пакетов.

5. Адаптация BBR к условиям пакетной балансировки

Проведённое экспериментальное исследование показало, что основная причина деградации пропускной способности BBR при пакетной балансировке связана с двумя эффектами: занижением оценки пропускной способности узкого места из-за потерь на медленных путях и снижением полезной скорости из-за повторных передач, вызванных переупорядочиванием пакетов при асимметрии задержек. При этом ключевую роль играет стандартный механизм `Fast Retransmit`, который при получении трёх дубликатных ACK (`DupAck`) инициирует повторную отправку предполагаемого потерянным пакета, даже если фактической потери не было, а пакет лишь задержался на более длинном пути.

В условиях асимметрии задержек пакет, попавший на канал с большей задержкой, может прибыть к получателю на величину разности задержек позже пакетов, отправленных по быстрому пути. За время этой задержки отправитель успевает получить от быстрых пакетов несколько `DupAck` и запускает повторную передачу. Объём данных, который может находиться “в полёте” без получения подтверждения по медленному пути, ограничен произведением пропускной способности узкого места на разность задержек. В рассмотренной топологии с двумя параллельными путями, имеющими задержки d_f и d_s ($d_s > d_f$), максимальное число пакетов, которые могут оказаться переупорядоченными, определяется величиной

$$N_{\max} = \min\left(\left\lceil \frac{\mathrm{BtlBw} \cdot (d_s - d_f)}{\mathrm{MSS}} \right\rceil, \mathrm{cwnd}\right)$$

где `MSS` — размер сегмента.

$$d_s \leq \frac{\mathrm{RTT}_{\max}}{2} \Rightarrow N_{\max} \leq \frac{\mathrm{RTT}_{\max} \cdot \mathrm{BtlBw}}{2 \cdot \mathrm{MSS}}$$

где $\mathrm{RTT}_{\max}$ - максимальный RTT пакета за некоторый период. Максимум достигается в ситуации, когда 1 пакет в окне попадает на маршрут с большей задержкой, а все остальные пакеты - на маршрут с меньшей задержкой. Следовательно, фиксированный порог в три `DupAck` оказывается непригодным: он многократно занижает допустимую степень переупорядочивания и провоцирует ложные повторные передачи, которые забивают канал дублирующими данными.

В качестве решения предлагается модифицировать алгоритм BBR, заменив фиксированный порог срабатывания `Fast Retransmit` динамическим значением:

$$\mathrm{DupAckThresh} = \min\left(\left\lfloor \frac{\mathrm{BtlBw} \cdot \mathrm{RTT}_{\max}}{\mathrm{MSS}} \right\rfloor, \alpha \cdot \mathrm{cwnd}\right), \alpha \in \left[\frac{1}{2}, 1\right)$$

Такая модификация позволяет отправителю ожидать прихода задержавшихся пакетов на интервале, сопоставимом с максимальным временем распространения по всем используемым путям. Предлагаемое изменение не затрагивает центральные механизмы BBR — оценку `BtlBw` и `RTprop` и управление скоростью через `pacing_rate`. Однако при $\mathrm{RTT}_{\max} \geq 2 \cdot \mathrm{RTprop}$ и $\alpha = 1$ скорость передачи данных будет ограничиваться `cwnd`, так как новые пакеты не будут отправлены в сеть, до получения Ack. Параметр α регулирет долю пакетов “в полете”, при получении DupAck. При $\alpha = \frac{1}{2}$, обьем данных “в полете” равен `BDP` не зависимо от полученных DupAck, так сохраняется базовый механизм управления потоком.

Литература

1. Dixit A. et al. On the Impact of Packet Spraying in Data Center Networks // IEEE INFOCOM. 2013. 9 с.
2. Cardwell N. et al. BBR: Congestion-Based Congestion Control // Queue. 2016. 34 с.
3. Gerstein B., Silberstein M., Keslassy I. Making congestion control robust to per-packet load balancing in datacenters // arXiv:2509.07907. 2025. 10 с.
4. Bhandarkar S. et al. Improving the Robustness of TCP to Non-Congestion Events. 2006. 18 с.
5. Zhang J., Zhang D., Huang K. Improving datacenter throughput and robustness with Lazy TCP over packet spraying // Computer Communications. 2015. 11 с.
6. Kleinrock L. Power and deterministic rules of thumb for probabilistic problems in computer communications // IEEE. 1979. 10 с.
7. Network simulator 3 [Электронный ресурс]. URL: https://www.nsnam.org/ (дата обращения: 27.04.2026).
8. Vivek Jain M. P. T. Viyom Mittal. Design and Implementation of TCP BBR in ns-3 // Workshop on ns-3, WNS3. 2018. 7 с.
9. GitHub проекта [Электронный ресурс]. URL: https://github.com/MikleMasterMind/diplom (дата обращения: 27.04.2026).
10. Павлович С. Е. Исследование методов многопоточной маршрутизации для обеспечения качества сетевого сервиса. 2022. 114 с.
11. Zhang S. An Evalution of BBR and its variants // In Proceedings of ACM Conference. 2019. 11 с.